Web Crawler - Step 1: Understand the Problem & Design Scope

1. Basic Crawling Algorithm

At a high level, a web crawler follows a simple iterative process:

Core Steps

1. Download web pages from a given set of URLs
2. Extract URLs from the downloaded pages
3. Add new URLs to the list and repeat

Flow Representation

```
None

URL List → Download Pages → Extract Links → Add New URLs → Repeat
```

2. Reality Check

Although the algorithm looks simple:

- Real-world web crawler design is **highly complex**
- Needs to handle:
    - Massive scale
    - Failures
    - Network issues
    - Duplicate data

3. Requirement Clarification

Purpose

Q: What is the crawler used for?
A: Search engine indexing

Scale

Q: How many pages per month?
A: ~1 billion pages

Content Type

Q: What type of content?
A: HTML only

Freshness

Q: Handle new/updated pages?
A: Yes

Storage

Q: Store crawled data?
A: Yes, up to 5 years

Duplicate Handling

Q: What about duplicate content?
A: Ignore duplicates

4. Key Requirements Summary

- Crawl **~1 billion pages/month**
- Support **HTML content**
- Handle **new + updated pages**
- Store data for **long-term (5 years)**
- Avoid **duplicate pages**

5. Characteristics of a Good Web Crawler

Scalability

- Web size = **billions of pages**
- Must support:
    - Parallel crawling
    - Distributed processing

Robustness

- Must handle:
    - Bad HTML
    - Broken links
    - Timeouts
    - Crashes
    - Malicious content

Politeness

- Avoid overwhelming websites
- Follow:
    - Rate limits
    - Crawl delays

Extensibility

- Easily support new features:
    - Images
    - PDFs
    - Videos
- No need for full redesign

6. Key Takeaways

- Simple algorithm ≠ simple system
- Real crawler design requires:
    - Scale handling
    - Fault tolerance
    - Smart scheduling
- Requirement clarity is critical before design